

Implementation Plan — `spatial_seas_driver.cpp`: General Quasi-Dynamic SEAS Driver for the SAF Mesh

CPU-only, assembled-once stiffness + iterative CG/GMRES–AMG solver, benchmarked against BP5

SEAS-MFEM-SAFS

2026-05-31

Contents

1 Implementation Plan: <code>spatial_seas_driver.cpp</code> — General Quasi-Dynamic SEAS Driver	2
1.1 Overview	2
1.2 Review remediation (2026-06-01 — addresses <code>spatial_seas_dev/REVIEW.md</code>) . . .	2
1.3 Review remediation (2026-06-02 — addresses <code>spatial_seas_dev/REVIEW_2026-06-02.md</code>)	3
1.4 Plan update (2026-06-03 — user scope changes)	3
1.5 Decisions locked with the user (2026-05-31)	4
1.6 Governing equations (the QD math the driver discretizes)	5
1.7 Architecture	5
1.8 File manifest — final structure (quasi-dynamic driver)	7
1.9 Constraints	11
1.10 Phase 0 — Build target, driver skeleton, dry-run	12
1.11 Phase 1 — QD config schema (<code>[solver]</code> , QD <code>[time]</code> , deprecate dynamic keys) . . .	13
1.12 Phase 2 — Mesh, boundary, material, domain operator (homogeneous-first)	14
1.13 Phase 2b — Heterogeneous material (REQUIRED; immediately follows Phase 2) . .	15
1.14 Phase 3 — Fault geometry + spatial per-DOF friction (the integration core)	16
1.15 Phase 3c — <code>bp5_analytic</code> stress source (BP5-parity prestress; R-007 / OBS-1) . . .	20
1.16 Phase 4 — Scalable iterative solver (CG/GMRES + BoomerAMG)	21
1.17 Phase 5 — SEAS operator, 4-phase init, adaptive RK45	22
1.18 Phase 6 — I/O: stations/probe, ParaView (fault), checkpoint/restart	24
1.19 Phase 7 — BP5 benchmark parity (the validation gate)	25
1.20 Phase 8 — SAF quasi-dynamic problem + Frontera jobs + scaling	26
1.21 Phase 9 — GPU-readiness (DESIGN ONLY — out of scope this round)	28
1.22 Testing Strategy	28
1.23 Risk Assessment	28
1.24 Open Questions (resolve before/at the noted phase)	30

1 Implementation Plan: `spatial_seas_driver.cpp` – General Quasi-Dynamic SEAS Driver

1.1 Overview

We add a **new, self-contained quasi-dynamic (QD) SEAS driver**, `drivers/spatial_seas_driver.cpp` (built as `seas_spatial_seas_driver`), that runs the spatially-heterogeneous San-Andreas-Fault (SAF / “SAFS”) problem under the **quasi-dynamic approximation** (quasi-static elasticity + radiation damping $\eta \mathbf{v}$, *no* wave propagation). It **mirrors the spatial scaffolding** of the dynamic driver `drivers/spatial_dyn_driver.cpp` (TOML config, parallel mesh load, spatially-varying material, fault geometry from a stress source, per-fault-DOF friction from `SpatialFrictionResolver`, ParaView/checkpoint I/O) but **drives the existing QD operator stack** that BP5 uses (`SEASQuasiDynamicOperator` $\rightarrow$ `ElasticityDomainOperator` + `RateStateFaultOperator` + `DieterichRuinaFriction/AgingLawPsi`), and it **replaces the BP5 direct MUMPS solve with an iterative CG/GMRES + algebraic-multigrid (HypreBoomerAMG) solver** so the $\sim 10^7$ -DOF SAF mesh is tractable. BP5 is the verification benchmark (Tandem is the secondary reference).

This document is the **contract** between the implementing agent and the reviewing agent. Function signatures, file paths, and acceptance criteria are normative.

1.2 Review remediation (2026-06-01 – addresses `spatial_seas_dev/REVIEW.md`)

The first plan review (verdict FAIL) found two CRITICAL plan-text defects and four lesser ones. All are now folded into the phases below; the IDs map to `REVIEW.md`:

- **R-001 (CRITICAL — construction order)**. `RateStateFaultOperator`’s BP5 ctor asserts `!geom.HasParams()` (`rate_state_fault.hpp:149`) **and** caches `Dc_values/V_init_values_` at construction (`:160–171`, never re-sourced by `SetSAFSMode`). The normative order is now **`geom(false)` $\rightarrow$ `SetRateStatePerDOF` $\rightarrow$ `fault-op ctor` $\rightarrow$ `ComputeParams*` $\rightarrow$ `SetSAFSMode`** (Phase 3 step 3 + Phase 5 snippet). `SetRateStatePerDOF` must not set `params_computed_`. The σ_n for `ResolveRateState` is sourced **independently** (Phase 3), not from `geom.sigma_n_per_dof()` (which is unpopulated until the deferred `ComputeParams*`).
- **R-002 (CRITICAL — owned-vs-local seam)**. `GetFaultDOFCoords3D` emits the **local** fault-DOF set (`num_fault_dofs_`), but the resolver/geometry/operator use the **owned** set (`num_owned_fault_dofs_`). Phase 3 now restricts coords with `RestrictToOwnedFault(..., 3)` before `ResolveRateState`; the new `GetFaultDOFToElem/Attr` getters are owned. (Serial masks this; `np>1` aborts.)
- **R-003 (MODERATE — σ_n total vs effective)**. `ResolveRateState`’s last arg is **total** σ_n (it subtracts pore pressure internally, `spatial_friction.cpp:2017`). Phase 3 now passes a total- σ_n vector (or size-0 + `sigma_n_default`), not the effective `geom.sigma_n_per_dof()`.
- **R-004 (MODERATE — RK45 API)**. Phase 5 now uses `SetAbsTol/SetRelTol` (no `SetTolerances`), `SetDtMax`, `SetMPIContext`, and `Step(seas_op, state, t, dt)` (operator-first).
- **R-005 (MODERATE — rs fields)**. Phase 5 now extracts scalar friction constants from the

uniform per-DOF vectors using the real field names `rs.V_0(0)/rs.f_0(0)/rs.b(0)/rs.Dc(0)`.

- **R-006 (LOW — SetElasticityOptions on DG)**. Phase 4 makes the residual check **mandatory** (default true) and flags the CFEM near-null-space-on-DG as unproven with a documented MUMPS fallback.

1.3 Review remediation (2026-06-02 – addresses spatial_seas_dev/REVIEW_2026-06-02.md)

A second, independent plan review re-verified R-001–R-006 (all correctly folded in above, verified against source) and found four **new** defects; all are now folded into the phases below:

- **R-007 (CRITICAL — BP5 prestress not reproducible by uniform FaultLocalPrestress)**. `ComputeBP5Params` never fills `sigma_n_per_dof_` (`fault_geometry.hpp:1391`; only `ComputeParamsFaultLocal`/the sidecar projection do), and BP5’s analytic `tau_pre_` is spatially heterogeneous (`bp5_params.hpp:333` `tau0_vec`: $a(x_2, x_3)$ -dependent + nucleation `delta_tau`). So a **uniform** `FaultLocalPrestress` cannot reproduce it and a size-0-vs-N `sigma_n_per_dof_` comparison is ill-posed. Phase 3’s parity assertion is now **split by who populates each array** (`a/Dc/eta/V_init` via the resolver to $1e-10$; τ_{pre}/σ_n via a per-DOF source), and Phase 7 now mandates a **per-DOF** BP5 prestress (a `bp5_analytic` stress source or a `tau0` sidecar) — not uniform `FaultLocalPrestress`.
- **R-008 (MODERATE — ParseQDSolverType return type)**. Phase 1 now returns `mfem::seas::SolverType` (the free enum, `elasticity_operator.hpp:46`), not `std::string`, so the Phase-2 ctor call type-checks.
- **R-009 (MODERATE — checkpoint dynamic-code dependency)**. Phase 6 now specifies a **QD-native** checkpoint; reusing `io/tpv104_checkpoint.hpp` is forbidden — it `#includes` `dynamic/fault_face_flux.hpp/DOFData` (a §Constraints “No dynamic-code dependency” violation) and is Q+DOFData-centric.
- **R-010 (MODERATE — V_init expansion)**. `SetRateStatePerDOF` now takes an explicit `init_vel_dir` and decomposes the size-N scalar `rs.V_init` into the 2N (dip, strike) interleave along it (matching `bp5_params::V_init_vec`); it must **not** reference `tau_pre_`, which is NaN until `ComputeParams*` runs in Phase 5.

1.4 Plan update (2026-06-03 – user scope changes)

Four user-directed scope changes, folded into the phases / file manifest below:

1. **Config & job layout**. QD configs live in `config/safs_qd/` and QD sbatch scripts in `jobs/safs_qd/` — distinct from the dynamic-rupture cases (which keep `config/*.toml` + `jobs/safs/`).
2. **Phase 2b is now REQUIRED**, implemented immediately after Phase 2 (no longer optional/deferred): spatially-varying λ, μ in the elasticity assembly. BP5 parity (Phase 7) still runs the **constant** path as a bit-for-bit regression guard.
3. **I/O uses VTU/PVD**, same as the dynamic-rupture drivers — `seas::ParaViewOutput` in **Vtu mode** (`mfem::ParaViewDataCollection`; per-rank `.vtu` + a `.pvd` index). **Do NOT use the VTKHDF backend** (`ParaViewHDFDataCollection` / `.vtkhdf`), even on `MFEM_USE_HDF5` builds.

4. **Performance is a first-class concern.** Instrument with **Caliper** (`general/annotation.hpp` `MFEM_PERF_FUNCTION` / `MFEM_PERF_SCOPE` / `MFEM_PERF_BEGIN/END`, no-ops unless built with `MFEM_USE_CALIPER=YES`) to emit a **perfgraph** / **runtime-report for small problems** when comparing solvers/preconditioners (Phase 4) and for SAF solver scaling (Phase 8); report wall-times. **Reduce redundant computation:** assemble **K** once (already specified) **and** build the AMG/KSP preconditioner **once** and reuse it across every RK stage/step (**K** never changes in QD); compute the static per-DOF fault tables once; re-solve **V** only from the accepted state. Reference: `document/caliper_perfgraph_dev/`.

1.5 Decisions locked with the user (2026-05-31)

#	Decision	Consequence for this plan
A	CPU-only. No partial assembly (PA). PA buys nothing on tetrahedra (no sum-factorization) and no DG-elasticity face PA integrator exists.	The stiffness matrix is assembled once (as today) and solved iteratively (CG/GMRES + BoomerAMG). No <code>AssemblyLevel::PARTIAL</code> , no matrix-free apply, no LOR.
B	No GPU work this round.	Keep the operator/kernel structure GPU-port-friendly (so <code>document/gpu_dev/ v1-v3</code> still applies later) but write zero device code. Design-only notes in §“Phase 9”.
C	BP5 is a reference to follow.	BP5 parity (reproduce <code>seas_driver.cpp</code> station output to the existing golden tolerance) is the validation gate before any SAF production run. Interpreted (and to be vetoed by the user if wrong) as:
[1]	Quasi-dynamic separate from dynamic, no code reuse.	spatial_seas_driver.cpp must not depend on the dynamic/wave code (<code>dynamic/wave_operator.*</code> , <code>dynamic/fault_face_flux.*</code> incl. its <code>DOFData</code> , <code>dynamic/godunov_flux.*</code> , ADER/RK wave steppers, PML). It reuses the existing quasi-dynamic operator stack (<code>solver/seas_operator.hpp</code> , <code>domain/elasticity_operator*</code> , <code>fault/rate_state_fault.hpp</code> , <code>friction/*</code>) — that stack is the quasi-dynamic code and is already independent of the wave code, and it is exactly what BP5 uses.

Why “matrix-free” is satisfied by the iterative-AMG path. The thing that does **not** scale on the SAF mesh is the **direct MUMPS factorization** (memory/flops blow up for 3D), not the assembly. The assembled `HypreParMatrix` is built **once** and reused across all RK stages/steps. Swapping the *solve* from MUMPS to **CG/GMRES + BoomerAMG** is the scalable, GPU-future-proof choice (HYPRE has CUDA/HIP backends; MUMPS has none). This matches MFEM `ex17p` (DG elasticity), Tandem’s `mg_cheby.cfg` aspiration, and this repo’s own `gpu_dev v3` recommendation (“keep assembly, flip `mumps`→`cg`, let BoomerAMG run [on-device]”). Literal MFEM-PA / matrix-free DG elasticity is explicitly out of scope (research-grade; see `document/gpu_dev/PA_and_custom_function_gpu_analysis.md`).

1.6 Governing equations (the QD math the driver discretizes)

Bulk — quasi-static linear elasticity (no inertia). On the 3-D domain Ω with the fault as an internal interface Γ_f :

$$\nabla \cdot \sigma(u) = 0 \quad \text{in } \Omega, \quad \sigma(u) = \lambda(\nabla \cdot u) I + \mu(\nabla u + \nabla u^T).$$

There is **no** ρu term — contrast the dynamic driver’s velocity-stress wave system.

DG discretization (SIPG / BR2). Vector `DG_FECollection(order, dim=3, GaussLobatto), vdim=3, Ordering::byNODES`. Volume term `int_Omega sigma(u):eps(v)` via `ElasticityIntegrator`; interior/boundary faces via `DGElasticityIPCombinedIntegrator (IP/SIPG, eps=-1)` **or** `DGElasticityBR2Integrator (BR2)`. The IP penalty **must** include the elasticity-tensor coupling (3-D), per `CLAUDE.md` (scalar-only penalty blows up during nucleation, debug v12–v13 H3). The assembled operator K is **SPD** (so CG is valid).

Fault slip as an interior Dirichlet jump. Fault slip $S=(S_{\text{dip}}, S_{\text{strike}})$ enters the elasticity solve only through the **right-hand side** as a prescribed displacement jump $[[u]]=S$ on Γ_f (no Schur complement, no fault-DOF condensation). Far-field plate loading enters as a true Dirichlet BC. Each RHS evaluation solves the full-volume $K u = f(S, t)$ once.

Traction recovery + radiation-damped friction balance. After the solve, the on-fault traction is $\tau_{qs} = (\sigma(u) \cdot n)$ projected into the fault-local ($n, t_1=\text{dip}, t_2=\text{strike}$) frame. The slip rate V is the root of the **regularized rate-and-state + radiation-damping** balance, solved **per fault DOF**:

$$\|\tau_{\text{pre}} + \tau_{qs}\| = \sigma_n^{\text{eff}} a \operatorname{asinh}\left(\frac{V}{2V_0} e^{\psi/a}\right) + \eta V, \quad \eta = \frac{\mu}{2c_s}.$$

V is found by **Brent in $\log_{10} V$** (never Newton — `CLAUDE.md/debug v1`), then decomposed **parallel to traction** ($V_{\text{vec}} = (V/\tau) \cdot \tau_{\text{vec}}$; antiparallel $\Rightarrow$ blowup, debug v8). State evolves by the **aging law** in ψ -space:

$$\dot{\psi} = \frac{bV_0}{D_c} \left(e^{(f_0 - \psi)/b} - \frac{V}{V_0} \right), \quad \psi = f_0 + b \ln \frac{V_0 \theta}{D_c}.$$

σ_n responds elastically: $\sigma_n^{\text{eff}} = \sigma_n^{\text{pre}} + (-T \cdot n)$ (Tandem convention; `SetElasticSigmaN(true)`).

State vector & time integration. The ODE state is the **fault only**: per fault DOF $[s_{\text{dip}}, s_{\text{strike}}, \psi]$ (`StatePerNode=3, SlipComponents=2`). $d(\text{slip})/dt = V$, $d\psi/dt$ as above. Displacement u is **not** a state variable — it is recomputed by the elasticity solve inside every RHS evaluation. Integration is **adaptive embedded Dormand–Prince RK45** (`atol=1e-7, rtol=1e-50, L $\infty$  error, MPI_Allreduce(MAX)` for consistent accept/reject; `dt_init=0.01*L_nuc/V_nuc, dt_max=0.1 yr`).

1.7 Architecture

What is reused as-is (no changes)

- `solver/seas_operator.hpp` — `SEASQuasiDynamicOperator<ParMesh, ElasticityDomainOperator<ParMesh>, RateStateFaultOperator<ParMesh, 2>>` (alias `PBP5SEASOp`). Generic over

domain/fault types; needs no change.

- `fault/rate_state_fault.hpp` — `RateStateFault0Operator<ParMesh,2>` and its `SetSAFS-Mode(...)` per-DOF σ_n/τ_{pre} hook.
- `friction/dieterich_ruina.hpp`, `friction/state_evolution.hpp` — friction + aging law (scalar $b/V0/f0$).
- `solver/time_stepper.hpp` — `DormandPrinceRK45` (CPU, no PETSc dependency).
- Spatial scaffolding: `spatial/code/spatial_friction.*` (config + `SpatialFriction-Resolver`), `spatial/code/spatial_stress.*`, `spatial/code/spatial_velocity.*`, `dynamic/heterogeneous_material.*` (`MaterialField` — NB: lives under dynamic/ but is a generic field abstraction, not wave code), `domain/boundary_config.hpp`, `io/paraview_output.hpp`, `common/mpi_context.hpp`, `fault/fault_basis.hpp`, `config/bp5_params.hpp`. (NB: `io/tpv104_checkpoint.hpp` is **not** reused — it depends on `dynamic/fault_face_flux.hpp/D0FData`; the QD checkpoint is new code, see Phase 6 / R-009.)

What is new code (the integration backbone)

1. **`drivers/spatial_seas_driver.cpp`** — the driver itself (the bulk of the work).
2. **`ElasticityDomain0Operator` getters** (`domain/elasticity_operator.hpp`): expose the per-fault-DOF `dof_to_elem`, `dof_to_attr`, and reference `IntegrationPoints` in the canonical **owned**-fault order (restricted via `RestrictToOwnedFault`), so `SpatialFrictionResolver::ResolveRateState(...)` can run on the QD fault DOFs **without** pulling in the dynamic driver's `BuildPerD0FFaultTables`. (The existing `GetFaultD0FCoords3D/GetFaultD0FBasis` are **local**-ordered — the driver must restrict the coords to owned before use; see R-002.)
3. **`FaultGeometry::SetRateStatePerD0F(const spatial::RateStatePerD0FParams&, const mfem::Vector& init_vel_dir)`** (`fault/fault_geometry.hpp`): fill the per-DOF `a_values_`, `dc_values_`, `eta_values_`, `V_init_vec_` from the spatial resolver output (today the spatial ctor NaN-fills these; only τ_{pre}/σ_n are set by `ComputeParams*`). `rs.V_init` is a **size-N scalar magnitude** (`spatial_friction.cpp:1941`); decompose it into the [2N] (`dip,strike`) interleaved **along the prescribed initial-velocity direction `init_vel_dir`**, matching `bp5_params::V_init_vec`'s convention (`V_init_vec_(2i)=dip`, `V_init_vec_(2i+1)=strike`). **Do NOT key the split off `tau_pre`** — it is unpopulated (NaN) until `ComputeParams*` runs in Phase 5, *after* this call (R-001 order; R-010). It must **not** set `params_computed_` (it runs before the fault-op ctor's `!HasParams()` guard; R-001).
4. **`FaultGeometry` domain-op ctor variant that skips analytic BP5 params** (`fault/fault_geometry.hpp`): so `FaultGeometry` is built over the **same owned/local layout** as BP5 (avoiding the owned-vs-local seam) but leaves the friction arrays for `ComputeParams*` + `SetRateStatePerD0F` to fill.
5. **Iterative-AMG solver wiring** (`domain/elasticity_operator*`): the `SolverType::CG_AMG` / `GMRES_AMG` paths already exist; this plan **tunes** them (`BoomerAMG SetElasticityOptions/SetSystemsOptions`, KSP tolerances/maxits, optional residual check) and exposes the knobs through config.
6. **Config additions** (`spatial/code/spatial_friction.{hpp,cpp}`): a [solver] table and QD [time] knobs; deprecate/ignore dynamic-only keys.
7. **Build + jobs**: Makefile target `seas_spatial_seas_driver`; Frontera jobs/`safs_qd/ sbatch` scripts (distinct from the dynamic-rupture jobs/`safs/`); QD configs under `config/safs_qd/`;

a BP5 verification TOML for the new driver.

The two enabling facts (verified in the codebase)

- **FaultGeometry<MeshType> is one shared class**; the spatial driver and the QD elasticity operator both enumerate fault DOFs by the **same face-walk + face-quadrature order** (interior fault faces, then shared fault faces; nbf_per_face nodal QPs each; identical Tandem t1=dip, t2=strike frame). So the spatial per-DOF arrays (a/b/Dc/tau_pre/sigma_n/...) line up **1:1** with the QD fault operator's $i = 0..NumNodes()-1$ — **provided the resolver inputs are restricted to the owned set** (the QD operator's num_nodes_ = GetNumOwnedFaultDOFs()). The face-walk getters (GetFaultDOFCoords3D etc.) produce the **local** set (GetNumFaultDOFs()); on np>1 owned < local, so the driver must RestrictToOwnedFault the coords before pairing them with the owned dof_to_elem/attr. (R-002)
- **ElasticityDomainOperator::SolverType already enumerates CG_AMG and GMRES_AMG** (domain/elasticity_operator.hpp:46) — the MUMPS→iterative switch is configuration + tuning, not new physics.

Critical-path call graph (per accepted RK45 step)

```
DormandPrinceRK45::Step
└─ SEASQuasiDynamicOperator::Mult(state, rate)          solver/seas_operator.hpp:317
    │ fault.GetSlip(state, slip)
    │ domain.ExpandOwnedToLocalFault(slip, local_slip)
    │ domain.Solve(t, local_slip, u)                    domain/elasticity_operator_traction.inl:394
    │ │ AssembleStiffness() [once; cached HyPreParMatrix] *** iterative CG/GMRES+BoomerAMG ***
    │ │ │ Assemble RHS: slip jump + Dirichlet plate loading
    │ │ │ └─ solver_→Mult(B, X)                          ← the scalable solve (this plan's core)
    │ │ domain.ComputeTraction(u, local_slip, τ, &σn_el) → RestrictToOwnedFault
    │ └─ fault.ComputeRHS(τ, state, rate, &σn_el)        fault/rate_state_fault.hpp:457
    │ │ └─ per owned DOF: Brent solve  $\tau = \sigma \cdot f(V, \psi) + \eta V$  ; rate[ψ]=aging
```

1.8 File manifest – final structure (quasi-dynamic driver)

A single consolidation of **every** file the QD effort creates or modifies, with the **dynamic-driver-impact** column the regression contract requires. This manifest reflects the **dynamic-safe target** — it incorporates the decisions of REVIEW_dyn_impact_2026-06-02.md (R-001, R-002): the dynamic-key warning lives only in the QD driver, and the shared StressSourceKind enum is **not** extended. (The Phase 1 §“Files to Modify” warning wording and the Phase 3c enum wording must be updated to match — see §“Dynamic-driver-safety guards” below.)

Legend — **NEW**: created · **MOD(add)**: modified, additive only (new symbols / overloads / struct members / key reads; no existing signature, default, or behavior changed) · **reuse**: compiled/linked unchanged.

A. New files (created)

```
miniapps/seas/
├─ drivers/
│   └─ spatial_seas_driver.cpp          NEW   Phases 0–6 – the driver (bulk of the work)
├─ io/
│   └─ seas_qd_checkpoint.hpp          NEW   Phase 6 – QD-native checkpoint (R-009; no
└─ DOFData)
```

— config/safs_qd/	(NEW subfolder – QD configs, distinct from
↪ dynamic-rupture config/*.toml)	
— bp5_spatial_seas_verification.toml	NEW Phase 7 – BP5 parity, [solver].type="mumps"
— bp5_spatial_seas_verification_cgamg.toml	NEW Phase 7 – sibling, cg_amg
— bp5_tau0_sidecar.h5 (optional)	NEW Phase 3c opt (ii) – per-DOF BP5 τ_0/σ_n sidecar
— safs_qd_500m.toml	NEW Phase 8 – SAF QD problem config
— jobs/safs_qd/	(NEW subfolder – QD sbatch, distinct from jobs/safs/
↪ = dynamic-rupture)	
— safs_qd_500m_dev_2hr.sbatch	NEW Phase 8 – Frontera dev smoke
— safs_qd_500m_production.sbatch	NEW Phase 8 – Frontera production
— tests/	
— unit/	
— test_spatial_seas_config.cpp	NEW Phase 1 – target seas_test_spatial_seas_config
— test_spatial_seas_faultgeom_parity.cpp	NEW Phase 3 – spatial path == analytic BP5 arrays
— test_spatial_seas_iterative_vs_direct.cpp	NEW Phase 4 – CG-AMG == MUMPS (u + traction)
— verification/	
— test_spatial_seas_bp5_parity.*	NEW Phase 7 – station parity vs seas_driver golden

B. Modified shared files (all additive)

File	Phase	Change	Affects spatial_dyn_driver?
Makefile	0,1,4,7	add seas_spatial_ seas_driver target (same OBJ prereqs as seas_driver , not the wave OBJs) + the new seas_test_* unit/verification targets	No — existing targets/rules untouched
spatial/code/spatial_ fric- tion.hpp / .cpp		add struct SolverSpec + SolverSpec solver; on SpatialFric- tionConfig; extend TimeSpec (rk45 tol + dt knobs); add mfem::seas::SolverType ParseQDSolver- Type(const std::string&); parse [solver] + QD [time] keys	No — new struct/members/fn + new key reads only; the shared parser's handling of existing numerics.* keys is unchanged (R-001)

File	Phase	Change	Affects spatial_dyn_driver?
domain/elasticity_operator.hpp (+_setup.inl, _trac-tion.inl)		add const getters GetFaultDOFTToElem / GetFaultDOFTToAttr / GetFaultDOFIntegrationPoints (owned order)	No — dyn driver never #includes or constructs ElasticityDomainOperator
domain/elasticity_operator_setup.inl /_assembly.inl		tune the existing CG-AMG/GMRES-AMG paths (BoomerAMG options, KSP tol/maxit, mandatory residual check); MUMPS/SuperLU/STRUMPACK paths untouched	No — dyn driver doesn't use this operator; BP5 uses the untouched MUMPS path
fault/fault_3_geometry.hpp		add 4-arg ctor (domain, seed, mpi, bool compute_bp5_params) — no default on the bool (R-003) — + void SetRateStatePerDOF(const RateStatePerDOFParams&, const Vector& init_vel_dir)	No — dyn driver uses the 7-arg prebuilt-array ctor; no-default bool keeps the 3-arg BP5 ctor unambiguous
spatial/code/spatial_stress.hpp		add class Bp5AnalyticStress (functor; evaluates bp5_params::tau0_vec/ σ n per DOF)	No — dyn driver never references this class; the shared StressSourceKind enum is NOT extended (R-002)

File	Phase	Change	Affects spatial_dyn_driver?
domain/elasticity_operator.hpp (+_assembly.in/_setup.in)	2b (required)	add coefficient-based ctor for heterogeneous λ, μ ; switch <code>lambda_coeff_</code> to <code>/mu_coeff_</code> to coefficient handles; thread into <code>ElasticityIntegrator</code> + IP/BR2 face integrators; constant path stays bit-for-bit	No — dyn driver doesn't use <code>ElasticityDomainOperator</code> ; the constant ctor is a bit-for-bit regression guard for BP5
drivers/spatial_seas_driver.cpp (+ optional do-main/elasticity_operator_*.inl)	4.8	additive MFEM_PERF_ SCOPE/MFEM_ PERF_FUNCTION Caliper annotations on assemble/solve (no-ops unless MFEM_USE_ CALIPER=YES)	No — macros expand to nothing without Caliper; zero behavior change; dyn driver unaffected

C. Dynamic-driver-safety guards (these MUST stay out of shared code)

1. **Dynamic-key “ignored” warning** → emitted by `spatial_seas_driver.cpp` only (after `LoadSpatialFrictionConfig`), never inside the shared `ParseSpatialFrictionConfigString/parse_root`. Otherwise the dynamic driver warns on its own valid numerics. `ader_order/mixed_flux/use_pml/...` keys (which the shared parser legitimately reads). **(R-001 — supersedes the Phase 1 line-231 wording.)**
2. **StressSourceKind enum is NOT extended.** BP5-analytic prestress is reached either by (i) the QD driver calling `geom.ComputeParams(Bp5AnalyticStressSource{bp5_params}, ...)` behind a QD-only `[stress].bp5_analytic=true` flag, or (ii) the existing `SidecarHDF5` kind + a precomputed τ_0 sidecar. Neither adds an enumerator, so `spatial_dyn_driver`'s catch-all else (`== SidecarHDF5`) dispatch is unchanged. **(R-002 — supersedes the Phase 3c “add StressSourceKind::Bp5Analytic” wording.)**

D. Reused unchanged (compiled/linked or read-only; zero edits)

`solver/seas_operator.hpp` (`SEASQuasiDynamicOperator`) · `fault/rate_state_fault.hpp` (`RateStateFaultOperator<ParMesh,2> + SetSAFSMode`) · `friction/dieterich_ruina.hpp` · `friction/state_evolution.hpp` · `solver/time_stepper.hpp` (`DormandPrinceRK45`) · `spatial/code/spatial_velocity.*` · `dynamic/heterogeneous_material.*` (`MaterialField`) · `domain/boundary_config.hpp` · `io/paraview_output.hpp` · `io/bp5_parallel_output.hpp` · `common/mpi_context.hpp` · `fault/fault_basis.hpp` · `config/bp5_params.hpp` (**read-only** — the new `Bp5AnalyticStressSource` calls `tau0_vec/a_of_x2_x3/σn`; no edits).

E. Explicitly NOT touched (regression contract / project memory [C2])

- Drivers: `drivers/seas_driver.cpp` (BP5), `drivers/spatial_dyn_driver.cpp` (dynamic), all `drivers/tpv*` — must build + make test unchanged.
 - Wave/dynamic stack: `dynamic/wave_operator.*`, `dynamic/fault_face_flux.*` (D0FData), `dynamic/godunov_flux.*`, `dynamic/rk_time_stepper.*`, `dynamic/pml_layer.*`, `io/tpv104_checkpoint.hpp` — **no #include, no link dependency** from the QD driver.
 - Shared `StressSourceKind` enum + `ParseSpatialFrictionConfigString`'s `numerics.*` handling — unchanged (§C).
-

1.9 Constraints

- **Interfaces that must NOT change** (regression contract): `seas_driver.cpp` (BP5), `spatial_dyn_driver.cpp` (dynamic), and all TPV* drivers must build and pass make test unchanged. New getters/ctors/config keys are **additive**; do not alter existing signatures or defaults. (CLAUDE.md: “Stiffness matrix assembled once and reused”; “Mix structural refactoring with numerical changes” is forbidden — keep each commit single-purpose.)
- **No dynamic-code dependency** in `spatial_seas_driver.cpp`: do **not** #include `dynamic/wave_operator.*`, `dynamic/fault_face_flux.*`, `dynamic/godunov_flux.*`, `dynamic/rk_time_stepper.*`, `dynamic/pml_layer.*`, or use `D0FData`. (Reuse `dynamic/heterogeneous_material.*` `MaterialField` only — it is a field abstraction, not wave physics; if the reviewer judges this a violation of [1], relocate `MaterialField` to a neutral path in a follow-up.)
- **Numerical invariants** (all from CLAUDE.md, learned debug v1–v62 — violating any causes failure):
 - Friction: **Brent**, not Newton; degenerate-bracket guard if $(F_b \geq 0) \ V = \tau / \eta$; no artificial τ floor.
 - Signs: slip-rate **parallel** to τ ; pre-stress parallel to V_i ; dip $(0, 0, +1)$; DG face sign $(\text{nor}(0) > 0) ? -1 : 1$; $\sigma_n > 0$ compression; $Z < 0$ depth; $t1 = \text{dip}$, $t2 = \text{strike}$.
 - BCs: **all non-free-surface boundaries get Dirichlet loading** (`BCMode::FarField`); free surface = Natural.
 - Time: $dt_{\text{init}} = 0.01 * L_{\text{nuc}} / V_{\text{nuc}}$; $dt_{\text{max}} = 0.1 \text{ yr}$; $atol = 1e-7$, $rtol = 1e-50$; `MPI_Allreduce(MAX)` error reduction.
 - Init: **4-phase** (`PreInit` $\rightarrow$ `Solve` $\rightarrow$ traction $\rightarrow$ Init ψ $\rightarrow$ verify equilibrium $< 1e-6$); per-DOF D_c in `PsiToTheta`.
- **Mesh**: Gmsh **v2.2 ASCII only** (`mfem::Mesh::ReadGmshMesh` has no v4 branch). SAF mesh is **tetrahedral**. Material **heterogeneity is implemented in Phase 2b (required)** — Phase 2 ships homogeneous-first, Phase 2b adds heterogeneous λ, μ ; the SAF mesh `..._500m_...trig.msh` is 62 MB / curved fault, $\sim 10^6$ DOF at p1.
- **I/O = VTU/PVD, not VTKHDF**: the QD driver uses `seas::ParaViewOutput` in `Vtu` mode (`mfem::ParaViewDataCollection`; per-rank `.vtu` + `.pvd`), like the dynamic-rupture drivers. Do **not** select the `ParaViewHDFDataCollection/.vtkhdf` backend even on `MFEM_USE_HDF5` builds. (Checkpoint is the separate QD-native format, Phase 6 / R-009.)
- **Performance & profiling**: instrument with **Caliper** (`general/annotation.hpp` `MFEM_PERF_*` macros; build `MFEM_USE_CALIPER=YES`, no-op otherwise) to emit a **perfgraph/runtime-report**

for small problems when comparing solvers/preconditioners (Phase 4) and SAF scaling (Phase 8); report wall-times. **Eliminate redundant computation:** K assembled once **and** AMG/KSP preconditioner built once and reused across all RK stages/steps; static per-DOF fault tables computed once; V re-solved only from the accepted state. Ref document/caliper_perfggraph_dev/.

- **MFEM_USE_MPI mandatory;** MFEM_DIR=../.. build-from-worktree caveat applies (see Phase 0).

1.10 Phase 0 – Build target, driver skeleton, dry-run

Goal

make seas_spatial_seas_driver builds an empty-but-wired driver that parses --config, prints a banner, and exits on --dry-run; existing builds/tests are unaffected.

Files to Create

- drivers/spatial_seas_driver.cpp — skeleton: MPI init, CLI parse (--config, --dry-run, --mesh, --tfinal, --restart, --checkpoint-every, the --paraview-* surface), TOML load via LoadSpatialFrictionConfig, rank-0 banner, MPI_Finalize. Copy the **CLI-helper + config-merge** structure from spatial_dyn_driver.cpp:134–200,504–756 (general scaffolding); **omit** all dynamic-only flags (--ader-order, --mixed-flux, --pml*, --time-integrator).

Files to Modify

- Makefile — add, mirroring the seas_spatial_dyn_driver block (Makefile:188, 2046–2059, 4482–4508):
 - SPATIAL_SEAS_DRIVER_SRC = drivers/spatial_seas_driver.cpp
 - SPATIAL_SEAS_DRIVER_OBJ = drivers/spatial_seas_driver.o
 - target seas_spatial_seas_driver: with the **same object prerequisites** as the QD/BP5 driver seas_driver (Makefile:4075) — i.e. the elasticity/fault/solver/friction/spatial OBJs, **not** the wave OBJs.

Detailed Requirements

1. CLI sentinels match the spatial driver: empty string / -1 / <0 ⇒ “keep TOML”; CLI overrides TOML.
2. --dry-run exits after config echo + derived-parameter print, before any operator construction.
3. Build-from-worktree: document in a header comment that building under .claude/worktrees/ needs make MFEM_DIR=../.. MFEM_BUILD_DIR=\$MAIN MFEM_INC_DIR=\$MAIN MFEM_LIB_DIR=\$MAIN seas_spatial_seas_driver (per project memory; MFEM_DIR ?= ../.. fails with no libmfem.a).

Acceptance Criteria

- ☐ conda activate mfem-dev && cd miniapps/seas && make seas_spatial_seas_driver succeeds.

- `./seas_spatial_seas_driver --config <bp5.toml> --dry-run` prints config and exits 0.
- `make all && make test` still pass (no regression to other drivers/tests).

Dependencies

- Depends on: nothing. Required by: all later phases.

1.11 Phase 1 – QD config schema ([solver], QD [time], deprecate dynamic keys)

Goal

The shared `SpatialFrictionConfig` carries everything the QD driver needs (solver choice + AMG/KSP knobs, RK45 tolerances, plate-loading rate), and ignores dynamic-only keys with a clear warning.

Files to Modify

- `spatial/code/spatial_friction.hpp / .cpp`:
 - Add struct `SolverSpec` { `std::string` type = "cg_amg"; double ksp_rtol = 1e-8; double ksp_atol = 0.0; int ksp_maxit = 500; bool amg_elasticity_options = true; int amg_print_level = 0; double blr_tol = 1e-10; bool residual_check = true; }; and a `SolverSpec` solver; member on `SpatialFrictionConfig`. (residual_check defaults **true** per R-006 — the AMG paths must verify convergence.)
 - Extend `TimeSpec` (`spatial_friction.hpp:~`) with QD knobs: double rk45_atol = 1e-7; double rk45_rtol = 1e-50; double dt_init = -1; /*<0 => derive 0.01*L_nuc/V_nuc*/ double dt_max_years = 0.1; double plate_rate_vp = -1; /*<0 => from rate_state*/ bool use_petsc_ts = false;
 - Parse [solver] and the new [time] keys in `ParseSpatialFrictionConfigString`.
 - Add `mfem::seas::SolverType` `ParseQDSolverType(const std::string&)` mapping "cg_amg"→`SolverType::CG_AMG`, "gmres_amg"→`SolverType::GMRES_AMG`, "gmres_ilu"→`SolverType::GMRES_BlockILU`, "mumps"→`SolverType::MUMPS`, "mumps_blr"→`SolverType::MUMPS_BLR`, "superlu"→`SolverType::SUPERLU`, "strumpack"→`SolverType::STRUMPACK`; **default SolverType::CG_AMG. Return the enum, not a string (R-008)** — Phase 2 passes the result straight into the `ElasticityDomainOperator` ctor's `SolverType` parameter (`SolverType` is a free enum in `mfem::seas`, `elasticity_operator.hpp:46`); a `std::string` return would not type-check.
 - On load, if any dynamic-only key is present (`numerics.ader_order`, `mixed_flux`, `interior_flux`, `use_pml`, `time_integrator`), emit **one** rank-0 warning "ignored by spatial_seas (quasi-dynamic) driver" and proceed.

Edge Cases

- `[solver].type = "mumps"` must remain selectable (small-mesh BP5 parity cross-check).
- Unknown `solver.type` ⇒ hard error at parse (don't silently default).

Acceptance Criteria

- New unit test `tests/unit/test_spatial_seas_config.cpp` (target `seas_test_spatial_seas_config`): parses a TOML with [solver] + QD [time], asserts every field; asserts un-

known solver.type throws; asserts a dynamic-only key triggers the warning path (not a throw).

- Existing seas_test_*config* tests still pass.

Dependencies

- Depends on: Phase 0. Required by: Phases 2–5.

1.12 Phase 2 – Mesh, boundary, material, domain operator (homogeneous-first)

Goal

The driver loads the SAF tet mesh in parallel, builds a BoundaryConfig and a (homogeneous) LinearElastic material, and constructs ElasticityDomainOperator<ParMesh> with the configured iterative solver — assembling the stiffness once without crashing.

Files to Modify

- drivers/spatial_seas_driver.cpp:
 1. Mesh: Mesh smesh(cfg.mesh.path); ParMesh pmesh(MPI_COMM_WORLD, smesh); pmesh.SetCurvature(cfg.mesh.order); (reuse spatial_dyn_driver.cpp:899–911). Assert pmesh.Dimension()==3.
 2. Boundary: build BoundaryConfig bc from [boundary] (reuse spatial_dyn_driver.cpp:923–937). For SAF the convention is **free surface (Natural) on the z=0 top, Dirichlet plate loading on far-field box walls, fault interior** (validated in Phase 8).
 3. Material: build MaterialField material(...) (reuse spatial_dyn_driver.cpp:942–1059). **Phase 2 ships the homogeneous path only:** for MaterialField::Mode::Constant, extract constant lambda,mu and construct LinearElastic le(lambda, mu). If the config requests a heterogeneous mode, **abort with a clear message** (“heterogeneous material handled in Phase 2b — the next, required phase”) — do **not** silently fall back (CLAUDE.md). (Phase 2b, immediately following, removes this abort and wires the coefficient ctor.)
 4. Domain operator:

```
ElasticityDomainOperator<ParMesh> domain(
    pmesh, cfg.mesh.order, le,
    /*Vp=*/plate_rate, /*Wf=*/fault_depth, /*lf=*/fault_length,
    bc, dg_method, ParseQDSolverType(cfg.solver.type), domain_config);
```

Vp/Wf/lf come from config (plate_rate from [time].plate_rate_vp or [friction.rate_state]).

Interfaces (existing, do not change)

- ElasticityDomainOperator(MeshType&, int order, const ConstitutiveModel&, real_t Vp, real_t Wf, real_t lf, const BoundaryConfig&, DGMethod, SolverType, const DomainConfig&={}) — domain/elasticity_operator.hpp:106.

Edge Cases

- MaterialField non-constant $\Rightarrow$ abort (Phase 2b territory).
- Mesh attribute set mismatch with [boundary] $\Rightarrow$ abort listing the present attributes.

Acceptance Criteria

- ☐ On the BP5 mesh (bp5/mesh/*.msh), the driver builds ElasticityDomainOperator and triggers stiffness assembly without abort, in serial and on mpirun -np 4.
- ☐ --dry-run after this phase prints GetNumFaultDOFs(), GetNumOwnedFaultDOFs(), GetNumFaultFaces(), GetNbfPerFace().

Dependencies

- Depends on: Phase 1. Required by: Phase 3.

1.13 Phase 2b – Heterogeneous material (REQUIRED; immediately follows Phase 2)

Goal

Spatially-varying λ, μ (depth profile / velocity sidecar) in the elasticity assembly. **This phase is required** (user decision 2026-06-03) — the SAF problem uses a velocity model. The constant path from Phase 2 must remain bit-for-bit identical (it is the BP5-parity regression guard).

Files to Modify

- domain/elasticity_operator.hpp + domain/elasticity_operator_assembly.inl + ..._setup.inl: add a ctor ElasticityDomainOperator(MeshType&, int order, mfem::Coefficient& lambda_c, mfem::Coefficient& mu_c, real_t Vp, real_t Wf, real_t lf, const BoundaryConfig&, DGMethod, SolverType, const DomainConfig&={}), switch the members lambda_coeff_/mu_coeff_ (elasticity_operator.hpp:456, currently ConstantCoefficient) to coefficient handles (own or non-own; non-owning, the coefficients must outlive the operator), and thread them into ElasticityIntegrator + the IP/BR2 face integrators (penalty must still include the elasticity-tensor coupling, CLAUDE.md).
- drivers/spatial_seas_driver.cpp: when material.mode() != Constant, build λ, μ Coefficients from MaterialField ({Lambda, Mu}FromSidecar / depth profile) and construct via the new coefficient ctor (removing the Phase-2 abort); when Constant, keep the existing LinearElastic ctor unchanged.
- **Note:** elasticity_operator.hpp is an “extreme care” file (CLAUDE.md) — this is its own commit with full elasticity-assembly tests; the constant ctor and all existing call sites (BP5/BP2/TPV*) stay unchanged (additive overload only).

Detailed Requirements

1. The new coefficient ctor and the existing constant ctor share one assembly path; the constant ctor wraps lambda, mu in ConstantCoefficient and forwards — so a single code path, no duplication.
2. MaterialField coefficient sources: depth profile (piecewise/continuous $\lambda(z), \mu(z)$) and a velocity-model sidecar; reuse the dynamic driver’s MaterialField accessors (no new field

abstraction).

3. Caliper-annotate the assembly so the heterogeneous-vs-constant assemble cost is measurable (Phase-4 perf concern).

Edge Cases

- Coefficient lifetime: store non-owning handles; assert non-null; the MaterialField-derived coefficients must outlive the operator (own them in the driver scope).
- A MaterialField::Mode::GridFunction source with a different FE space than the elasticity space $\Rightarrow$ project/interpolate or abort with a clear message (do not silently mis-sample).

Acceptance Criteria

- ☐ Constant-coefficient path reproduces Phase 2 results **bit-for-bit** (regression guard; required before merge).
- ☐ A 2-layer depth profile yields the analytically expected traction on a 1-element-thick slab test.
- ☐ BP5 parity (Phase 7) still passes via the **constant** path (no regression from the coefficient refactor).

Dependencies

- Depends on: Phase 2. Required by: Phase 8 (SAF production uses heterogeneous material). BP5 parity (Phase 7) uses the constant path, so 2b is not a *blocker* for Phase 7, but it **is** required for the SAF run and is implemented now (not deferred).

1.14 Phase 3 – Fault geometry + spatial per-DOF friction (the integration core)

Goal

Build a FaultGeometry over the **same owned-fault layout** the QD stack expects, populate its per-DOF pre-stress/ σ_n (from the stress source) and rate-state params (from SpatialFriction-Resolver), and verify the arrays match the analytic BP5 path when fed BP5-equivalent inputs.

Files to Modify

1. **domain/elasticity_operator.hpp** (+ **..._setup.inl** / **..._traction.inl**) — add **const getters** that emit per-fault-DOF data in the canonical **owned** order (the order RestrictToOwnedFault produces), so the resolver can run on QD fault DOFs:

```
void GetFaultDOFToElem(mfem::Array<int>& dof_to_elem) const; // Elem1No per owned
↪ fault DOF
void GetFaultDOFToAttr(mfem::Array<int>& dof_to_attr) const; // face bdr/material
↪ attr per owned fault DOF
void GetFaultDOFIntegrationPoints(std::vector<mfem::IntegrationPoint>& ips) const;
↪ // ref IP per owned fault DOF
```

These walk the **same** `fault_interior_faces_` then `fault_shared_faces_` with `nbf_per_face_` QPs that `SetupFaultInfo` already uses (`elasticity_operator_setup.inl:633-713`), then **restrict to the owned set via RestrictToOwnedFault** so their length is `GetNumOwnedFaultDOFs()`. **WARNING (R-002):** the existing `GetFaultDOFCoords3D/GetFaultDOFBasis` emit the **LOCAL** set (size `GetNumFaultDOFs()`; interior + ALL shared faces) — they are *not*

owned-ordered and must be restricted (RestrictToOwnedFault(.., 3) for coords) before being paired with these getters. On np=1 local==owned, which masks the bug; on np>1 they differ and the resolver asserts.

2. `fault/fault_geometry.hpp` — add:

- A domain-op ctor variant that **skips analytic BP5 params**: `FaultGeometry(DomainOperator<MeshType>& domain, const BP5Params& seed, MPIContext* mpi, bool compute_bp5_params);` when `compute_bp5_params==false` it builds the **owned** per-DOF coords/basis/elem exactly as the existing (domain, params, mpi) ctor (`fault_geometry.hpp:118–160`) but does **not** call `ComputeBP5Params()` (leaves `a_/dc_/eta_/V_init_/tau_pre_/sigma_n_` for later fill). This guarantees the **owned↔local layout matches BP5** (no seam).
- `void SetRateStatePerDOF(const spatial::RateStatePerDOFParams& rs, const mfem::Vector& init_vel_dir);` — fills `a_values_`, `dc_values_`, `eta_values_`, and `V_init_vec_`, and `sigma_n_per_dof_` from `rs.sigma_n_eff` if not already set by a stress source. **V_init expansion (R-010)**: `rs.V_init` is a **size-N scalar magnitude** (the resolver writes one value per DOF, `spatial_friction.cpp:1941`); the fault op requires a **size-2N** `V_init_vec_` (`rate_state_fault.hpp:161` asserts `V_init.Size()==2*num_nodes_`). Decompose the magnitude into (dip,strike) **along init_vel_dir** (the prescribed initial-velocity / plate-loading direction — unit strike for BP5), matching `bp5_params::V_init_vec_`'s layout (`V_init_vec_(2i)=dip`, `V_init_vec_(2i+1)=strike`). **Do NOT base the split on tau_pre_**: per the R-001 order this method runs *before* `ComputeParams*`, so `tau_pre_` is still NaN here (`fault_geometry.hpp:286–287`). Asserts `rs.a.Size()==NumFaultDOFs()` (the **owned** count) and `init_vel_dir.Size()==2`. **MUST NOT set params_computed_** — this method runs *before* the `RateStateFaultOperator` ctor (which caches `Dc_values_/V_init_values_`), and the ctor asserts `!geom.HasParams()`; setting `params_computed_` here would trip that guard. (R-001) `init_vel_dir` is a single **global** (dip,strike) direction — correct for BP5's globally strike-slip loading; a *per-DOF* loading direction for the curved SAF fault is deferred to Phase 8 (see Open Questions #1). (OBS-2)

3. `drivers/spatial_seas_driver.cpp` — wire it. Order is normative (R-001/R-002/R-003):

```
MPIContext mpi(MPI_COMM_WORLD);           // owns_comm=false
BP5Params seed;                           // defaults; seeds only  $c_s/\mu \rightarrow \eta$ 
    ↪ scaling
FaultGeometry<ParMesh> geom(domain, seed, &mpi, /*compute_bp5_params=*/false);

// (1) OWNED-order per-DOF tables for the resolver. GetFaultDOFTToElem/Attr are
//     the NEW owned-order getters (Phase 3 step 1). The EXISTING
//     GetFaultDOFCoords3D emits the LOCAL set (size 3*GetNumFaultDOFs(),
//     interior + ALL shared faces), so it MUST be restricted to the owned set
//     with RestrictToOwnedFault — otherwise on np>1 it mismatches dof_to_elem
//     (owned) and ResolveRateState/SetRateStatePerDOF abort. (R-002)
Array<int> dof_to_elem, dof_to_attr;
domain.GetFaultDOFTToElem(dof_to_elem);    // size = GetNumOwnedFaultDOFs()
```

```

domain.GetFaultDOFToAttr(dof_to_attr); // size = GetNumOwnedFaultDOFs()
Vector local_coords_3d; domain.GetFaultDOFCoords3D(local_coords_3d); //
↳ LOCAL
Vector dof_coords_3d; domain.RestrictToOwnedFault(local_coords_3d, dof_coords_3d,
↳ /*comps=*/3); // OWNED

// (2) TOTAL effective-normal-stress per owned DOF, sourced INDEPENDENTLY of geom.
// The stress source (geom.ComputeParams*) is DEFERRED to Phase 5 – it must
// run AFTER the RateStateFaultOperator ctor (R-001), so geom.sigma_n_per_dof()
// is not yet populated here. For uniform  $\sigma_n$  (BP5 parity) leave this empty
// (size 0) and set [friction.rate_state].sigma_n_default; for a depth/sidecar
//  $\sigma_n$  model fill sigma_n_total_owned[N_owned] with TOTAL (pre-pore-pressure)
↳  $\sigma_n$ 
// evaluated at dof_coords_3d (a thin helper mirroring the  $\sigma_n$  branch of
// ComputeParams* but writing a plain Vector, NOT setting params_computed_).
Vector sigma_n_total_owned; // size 0 (uniform) OR size N_owned (TOTAL  $\sigma_n$ )

// (3) Resolve per-DOF rate-state. The LAST arg is the TOTAL normal stress
// (ResolveRateState subtracts pore pressure internally via `pp`). Do NOT
// pass geom.sigma_n_per_dof(): after ComputeParamsFaultLocal that holds the
// EFFECTIVE  $\sigma_n$ , which would be pore-pressure-subtracted a second time. (R-003)
spatial::SpatialFrictionResolver resolver;
spatial::PorePressureSpec pp; // from [stress.pore_pressure]; default-empty for BP5
spatial::RateStatePerDOFParams rs = resolver.ResolveRateState(
    *cfg.rate_state, dof_coords_3d, dof_to_elem, dof_to_attr,
    material, pmesh, pp, sigma_n_total_owned);

// (4) Fill geom's per-DOF a/Dc/eta/V_init. MUST run BEFORE the fault-operator
// ctor (the ctor caches Dc_values_/V_init_values_ and never re-sources them)
// and MUST NOT set params_computed_ (the ctor asserts !geom.HasParams()).
↳ (R-001)
// init_vel_dir is the prescribed initial-velocity / plate-loading unit
// direction (dip,strike); SetRateStatePerDOF decomposes the scalar
// rs.V_init(i) along it into V_init_vec_(2i)=dip, V_init_vec_(2i+1)=strike
// (matching bp5_params::V_init_vec). Do NOT use tau_pre here – it is NaN
// until ComputeParams* runs in Phase 5. (R-010)
Vector init_vel_dir(2); init_vel_dir(0) = 0.0; init_vel_dir(1) = 1.0; // BP5: pure
↳ strike
geom.SetRateStatePerDOF(rs, init_vel_dir);
// The stress source (geom.ComputeParams*/ComputeParamsFaultLocal – fills geom's
// tau_pre_/sigma_n_per_dof_ AND sets params_computed_=true) is intentionally
// deferred to Phase 5, AFTER the RateStateFaultOperator ctor.

```

Interfaces (existing, do not change)

- `SpatialFrictionResolver::ResolveRateState(const RateStateBlock&, const Vector& dof_coords_3d, const Array<int>& dof_to_elem, const Array<int>& dof_to_attr, const MaterialField&, ParMesh&, const PorePressureSpec&, const Vector& sigma_n_total_per_dof) const` — `spatial_friction.hpp:702`. **The last arg is TOTAL normal stress** (the resolver forms $\sigma_n_{\text{eff}} = \sigma_n_{\text{total}} - P_p$ internally, `spatial_friction.cpp:2017`);

it may be size 0 (then `cfg.sigma_n_default` is used) or size `N_owned`. **Do not pass effective σ_n** (R-003).

- `RateStatePerDOFParams { Vector a, b, Dc, V_init, f_0, V_0, eta, sigma_n_eff; Vector V_w; }` — `spatial_friction.hpp:687`.

Edge Cases

- **Spatially-varying $b/f_0/V_0$** : the QD friction law treats $b/V_0/f_0$ as **scalars** baked into `DieterichRuinaFriction/AgingLawPsi`. After `ResolveRateState`, **assert `rs.b`, `rs.f_0`, `rs.V_0` are uniform** ($\max - \min < 1e-12 \cdot \text{mean}$); if not, **abort** with “per-DOF $b/f_0/V_0$ not yet supported by the QD fault operator (see Phase 3b)”. Do not silently average.
- **Fallback guard for `SetSAFSMode`**: refuse if `geom.NumZeroNormalFallbacks() != 0` || `geom.NumT1Fallbacks() != 0` (degenerate fault normals) — these would corrupt the per-DOF $\tau_{\text{pre}}/\sigma_n$ mapping.
- **Owned-vs-local (R-002)**: `dof_to_elem/attr/coords` fed to `ResolveRateState` **must** be the owned set in owned order (matching `rs` indexing and `FaultGeometry i=0..N-1`). The new `GetFaultDOFToElem/Attr` getters are owned; `GetFaultDOFCoords3D` is **local** and must be passed through `RestrictToOwnedFault(..., 3)` first. Verify `dof_to_elem.Size() == dof_coords_3d.Size()/3 == GetNumOwnedFaultDOFs()` before calling the resolver.

Acceptance Criteria

- **Parity test** `tests/unit/test_spatial_seas_faultgeom_parity.cpp`: on the BP5 mesh, build `FaultGeometry` two ways — (a) the analytic BP5 ctor (`domain, params, &mpi`); (b) the new path (`domain, seed, &mpi, false`) + `SetRateStatePerDOF(rs, init_vel_dir) + ComputeParams*`, with a BP5-equivalent `[friction.rate_state]` TOML, **owned-restricted coords**, and **total σ_n** fed to `ResolveRateState`. **Split the comparison by who populates each array (R-007)**:
 - **`a_values_`, `dc_values_`, `eta_values_`, `V_init_vec_`** — populated by both `ComputeBP5Params` (path a) and `SetRateStatePerDOF(rs, init_vel_dir)` (path b); the resolver path **must** reproduce these to $1e-10$ element-wise. (For `eta_values_`: BP5 `eta` is auto-computed from material μ/c_s when `eta_auto` is set, `spatial_friction.cpp:2024`, whereas path (a) uses `seed's  $\mu/c_s$` — so the test `MaterialField  $\mu/c_s$  must equal` the BP5 seed values, else this parity fails for a benign reason. For `V_init_vec_`: `init_vel_dir` in path (b) must match `bp5_params::V_init_vec's` direction convention, R-010.)
 - **`tau_pre_`, `sigma_n_per_dof_`** — **`ComputeBP5Params` does NOT populate `sigma_n_per_dof_`** (`fault_geometry.hpp:1391`; only `ComputeParamsFaultLocal` / the sidecar projection write it), and the BP5 `tau_pre_` it *does* produce is **spatially heterogeneous** (`bp5_params.hpp:333 tau0_vec: a(x2,x3)-dependent + nucleation delta_tau`). So a **uniform** `FaultLocalPrestress` cannot match path (a), and a size-0-vs-size-N `sigma_n_per_dof_` comparison is **ill-posed**. Therefore either (i) drive **both** paths through the **same per-DOF prestress source** — the `bp5_analytic` stress source / `tau0 SidecarHDF5` of Phase 7 — and only then assert `path(a) == path(b)` to $1e-10$; or (ii) assert `path(b)'s tau_pre_/sigma_n_per_dof_` directly against `bp5_params::tau0_vec/ $\sigma_n$` evaluated on the owned DOFs. Do **not** assert element-wise equality of a uniform-`FaultLocalPrestress tau_pre_` against the analytic one.

(This test builds only `FaultGeometry` — no `RateStateFaultOperator` — so it is not subject

to the ctor `!HasParams()` guard; nonetheless use the production owned-coords + total- σ_n inputs so the test exercises the real wiring. Proves the spatial $\rightarrow$ QD wiring + DOF ordering.)

□ `mpirun -np 4` parity test passes (owned/shared layout consistent).

Dependencies

- Depends on: Phase 2. Required by: Phases 4, 5, 7.
-

1.15 Phase 3c – bp5_analytic stress source (BP5-parity prestress; R-007 / OBS-1)

Goal

Provide a **per-DOF** stress source that reproduces BP5's analytic steady-state prestress so the Phase-3 parity test and the Phase-7 gate can match `seas_driver.cpp`. Uniform `FaultLocalPrestress` **cannot** (BP5 `tau0` is $a(z)$ -heterogeneous + carries a nucleation `delta_tau`; R-007). This is small, additive, and on the BP5-parity critical path; it is **verification-only** (SAF production uses depth/sidecar prestress).

Files to Modify

- `spatial/code/spatial_stress.hpp`: add class `Bp5AnalyticStressSource` — a functor primitive mirroring `ConstantTensorStressSource`, but per fault DOF it evaluates `bp5_params::tau0_vec(x2, x3, tau)` (dip,strike) and the BP5 effective σ_n . Constructed from a `BP5Params` (the same spatial functions `ComputeBP5Params/seas_driver.cpp` use — `config/bp5_params.hpp:333`).
- `spatial/code/spatial_friction.{hpp,cpp}`: add `StressSourceKind::Bp5Analytic` and parse `[stress].kind = "bp5_analytic"` (no extra fields — it pulls the BP5 functions from `bp5_params.hpp`).
- `drivers/spatial_seas_driver.cpp` (Phase 5 §B dispatch): add the branch `Bp5Analytic` $\rightarrow$ `geom.ComputeParams(Bp5AnalyticStressSource{bp5_params}, ...)` alongside the existing `ConstantTensor/DepthProportional` functor dispatch. Like the other `ComputeParams*` paths it writes only `tau_pre_/sigma_n_per_dof_` (+ `params_computed=true`) and **preserves** the resolver-filled `a/dc/eta/V_init` (`fault_geometry.hpp:673` guard), so the R-001 construction order is unchanged.

Edge Cases

- Refuse `bp5_analytic` on a non-BP5 `FaultGeometry` (`!geom.IsBP5()`) — it relies on BP5 2D fault functions.
- Verification-only: do **not** offer it as a SAF production prestress.

Acceptance Criteria

- `Bp5AnalyticStressSource` fed BP5 params reproduces `bp5_params::tau0_vec` per owned DOF to round-off; the resulting `tau_pre_` matches the analytic-ctor `tau_pre_` to $1e-10$ element-wise — this closes the `tau_pre_/sigma_n_per_dof_` half of the Phase-3 parity test (R-007).

Dependencies

- Depends on: Phase 3. Required by: Phase 7 (BP5 prestress). Additive; off the SAF production path.

1.16 Phase 4 – Scalable iterative solver (CG/GMRES + BoomerAMG)

Goal

The per-step elasticity solve uses **CG (or GMRES) preconditioned by HypreBoomerAMG with elasticity options**, converging to a tolerance that reproduces the MUMPS displacement/traction, and scaling to the SAF mesh.

Files to Modify

- domain/elasticity_operator_setup.inl (SetupSolver) / ..._assembly.inl (solver dispatch ~:315–429):
 - CG_AMG: CGSolver + HypreBoomerAMG with SetSystemsOptions(dim=3, /*order_bynodes=*/true) and (when SPD/contiguous) SetElasticityOptions(fespace); set rel_tol=cfg.solver.ksp_rtol, abs_tol=ksp_atol, max_iter=ksp_maxit, print_level=amg_print_level. **Caveat (R-006)**: SetElasticityOptions builds CFEM rigid-body near-null-space modes; applying them to a **DG** space is unproven (MFEM ex17p does not). It will not crash (SetSystemsOptions/SetElasticityOptions handle byNODES, hypre.cpp:5192–5212), but if AMG iteration counts grow with h, drop to SetSystemsOptions-only or MUMPS.
 - GMRES_AMG: GMRESSolver + HypreBoomerAMG (no elasticity near-null-space for the non-CG path; DG sparsity breaks the CFEM assumption — keep SetSystemsOptions only).
 - **Mandatory** residual check on the AMG paths (cfg.solver.residual_check defaults **true**): compute $\|Kx-b\|/\|b\|$ and warn **loudly** with the iteration count if $> 10*ksp_rtol$ — a silently non-converged solve corrupts traction → friction → blowup. (R-006)
 - **Keep MUMPS/SuperLU/STRUMPACK paths untouched** (BP5 cross-check + small meshes).
 - **Build the solver + preconditioner ONCE; reuse across all RK stages/steps (no redundant recompute)**. K is assembled once and **never changes** in QD, so the HypreBoomerAMG setup and the CG/GMRES operator must be constructed once (at SetupSolver) and reused for every solver_→Mult(B,X) — do **not** call SetOperator/rebuild AMG per solve. Only B (the slip-jump + Dirichlet RHS) changes per evaluation. Verify the AMG setup phase runs exactly once (Caliper region count == 1).
- drivers/spatial_seas_driver.cpp: pass cfg.solver.* into the operator (extend DomainConfig if needed, additively).
- **Caliper profiling (build MFEM_USE_CALIPER=YES)**: annotate the assemble + per-solve regions with MFEM_PERF_SCOPE("...") / MFEM_PERF_FUNCTION (general/annotation.hpp; no ops without Caliper). For the small-BP5 solver/preconditioner comparison, emit a Caliper **runtime-report / perfgraph** (region tree with inclusive/exclusive times) via a CALI_CONFIG/cali::ConfigManager runtime-report, and report AMG-setup time + mean per-solve wall-time + iteration count for each variant (cg_amg, gmres_amg, mumps). See document/caliper_perfgraph_dev/.

Interfaces (existing)

- `HyprBoomerAMG::SetSystemsOptions(int dim, bool order_bynodes), SetElasticityOptions(ParFiniteElementSpace*)` — `linalg/hypr.hpp:1776–1785`. Require an **assembled HyprParMatrix** (already produced by `AssembleStiffness`).

Edge Cases

- AMG non-convergence within `ksp_maxit` $\Rightarrow$ **do not** silently return; warn loudly with residual + iteration count (a diverging solve corrupts traction $\rightarrow$ friction $\rightarrow$ blowup).
- The SEAS DG space is `Ordering::byNODES` (`elasticity_operator_setup.inl:13`); pass `order_bynodes=true` to `SetSystemsOptions` (MFEM defaults to `byVDIM`, `hypr.cpp:5196`). Assert `fes->GetOrdering()==Ordering::byNODES` so the AMG DOF-function mapping is correct. (`SetElasticityOptions` itself handles either ordering — it is a DG-suitability concern, not an ordering one; see R-006.)
- BR2 vs IP: both assemble SPD K; verify CG works for the chosen `dg_method` (BR2 default).

Acceptance Criteria

- ☐ **Solver-equivalence test** `tests/unit/test_spatial_seas_iterative_vs_direct.cpp`: on a small BP5 mesh, for a fixed slip BC, assert $||u_{CGAMG} - u_{MUMPS}|| / ||u_{MUMPS}|| < 1e-7$ and the recovered fault traction agrees $< 1e-7$ (relative). Run serial + `np=4`.
- ☐ AMG iteration count is reported and **bounded (h-independent within a factor)** across two BP5 mesh resolutions (smoke check of scalability, not a hard bound).
- ☐ **Preconditioner reuse**: over an N-solve sequence the AMG setup runs **once** (Caliper region count == 1 / a setup-time counter increments once), not per solve.
- ☐ **Perfgraph + speed report**: a Caliper runtime-report (perfgraph) is produced on a small BP5 mesh comparing `cg_amg / gmres_amg / mumps`, reporting AMG-setup time, mean per-solve wall-time, and iteration count; the chosen default (`cg_amg`) is justified by the report. (MFEM_USE_CALIPER=YES profiling build; functional builds are unaffected since the macros are no-ops.)

Dependencies

- Depends on: Phase 3. Required by: Phases 5, 7.

1.17 Phase 5 – SEAS operator, 4-phase init, adaptive RK45

Goal

Assemble the full QD coupling and integrate in time with adaptive RK45; a short BP5 run advances stably.

Files to Modify

- `drivers/spatial_seas_driver.cpp`. **Construction order is normative (R-001):** `geom(false)` $\rightarrow$ **Phase-3** `SetRateStatePerDOF` $\rightarrow$ **fault-op ctor** $\rightarrow$ `ComputeParams*` $\rightarrow$ `SetSAFSMode`. The ctor asserts `!geom.HasParams()`, so the stress source **MUST** run *after* it.

```
// Uniform-scalar friction constants from the (Phase-3 asserted uniform) per-DOF
// vectors. Fields are V_0 / f_0 (underscored) and are Vectors – take element 0.
```



```

// Per-DOF Dc still flows through geom's Dc_values_; fc.Dc is only the scalar
↪ fallback. (R-005)
MFEM_VERIFY(rs.b.Size() > 0, "empty rate-state");
DieterichRuinaFriction::Constants fc{ /*V0=*/rs.V_0(0), /*f0=*/rs.f_0(0),
                                     /*b=*/rs.b(0),    /*Dc=*/rs.Dc(0) };

DieterichRuinaFriction friction(fc);
AgingLawPsi aging(fc.b, fc.V0, fc.f0);

// (A) Construct the fault operator BEFORE the stress source (R-001). The BP5
//      ctor asserts !geom.HasParams() and caches Dc_values_/V_init_values_ from
//      geom (already filled by Phase-3 SetRateStatePerDOF).
RateStateFaultOperator<ParMesh,2> fault_op(&geom, &friction, &aging, seed, &mpi);

// (B) NOW run the stress source (sets geom's tau_pre_/sigma_n_per_dof_ +
//      params_computed_). Dispatch mirrors spatial_dyn_driver.cpp:1337-1458:
//      FaultLocalPrestress → geom.ComputeParamsFaultLocal(tau_strike, tau_dip,
↪ sigma_n, P_p)
//      ConstantTensor      → geom.ComputeParams(ConstantTensorStressSource{...},
↪ ...)
//      DepthProportional / SidecarHDF5 as in the dynamic driver.
//      The  $\sigma_n$  model here MUST be the same one whose TOTAL  $\sigma_n$  fed ResolveRateState in
↪ Phase 3.
//      For BP5 PARITY the prestress MUST be PER-DOF (a `bp5_analytic` source or a tau0
//      SidecarHDF5), NOT uniform FaultLocalPrestress – BP5's tau0 is a(z)-heterogeneous
//      and carries a nucleation-zone delta_tau (bp5_params.hpp:333; R-007 / Phase 7).

// (C) Point the fault operator at geom's per-DOF  $\tau_{pre}$  / effective  $\sigma_n$ .
fault_op.SetSAFSMode(true, &geom.GetTauPre(), &geom.sigma_n_per_dof()); // per-DOF
↪  $\sigma_n / \tau_{pre}$ 

PBP5SEASOp seas_op(&domain, &fault_op, &mpi);
seas_op.SetElasticSigmaN(true);
Vector state(fault_op.StateSize());
seas_op.SetInitialCondition(state); // 4-phase; aborts if equilibrium > 1e-6

DormandPrinceRK45 ode; ode.Init(seas_op);
ode.SetAbsTol(cfg.time.rk45_atol); // NB: SetAbsTol/SetRelTol – there is NO
↪ SetTolerances (R-004)
ode.SetRelTol(cfg.time.rk45_rtol);
ode.SetDtMax(cfg.time.dt_max_years * seconds_per_year); // internal cap; no manual
↪ clamp
ode.SetMPIContext(&mpi); // MPI_Allreduce(MAX) error reduction
↪ (CLAUDE.md)
double t = 0.0;
double dt = (cfg.time.dt_init > 0) ? cfg.time.dt_init : 0.01*L_nuc/V_nuc;
// loop: ode.Step(seas_op, state, t, dt); // the operator is the FIRST argument
↪ (R-004)

```

- **Nucleation:** BP5/SAF QD nucleation is the **initial-condition / loading** (no gradual_over-stress sub-step accumulator — that is the *dynamic* driver's mechanism and is off-limits here).

Seed the nucleation patch through [friction.rate_state] (elevated a-b, V_init) and/or a [nucleation] IC perturbation resolved into state/tau_pre at t=0. Specify the exact SAF nucleation in Phase 8.

Interfaces (existing, do not change)

- SEASQuasiDynamicOperator(DomainOpType*, FaultOpType*, MPIContext*), SetInitialCondition(Vector&), SetElasticSigmaN(bool), Mult(const Vector&, Vector&) — solver/seas_operator.hpp.
- RateStateFaultOperator(FaultGeometry*, DieterichRuinaFriction*, StateEvolution*, const BP5Params&, MPIContext*), SetSAFSMode(bool, const Vector*, const Vector*), StateSize(), GetSlip(...) — fault/rate_state_fault.hpp. **Ctor precondition (rate_state_fault.hpp:149): !geom.HasParams()** — construct before geom.ComputeParams* (R-001). The ctor caches Dc_values_/V_init_values_ from geom (rate_state_fault.hpp:160-171), so SetRateStatePerDOF must precede it.
- DormandPrinceRK45 — solver/time_stepper.hpp. API: Init(TimeDependentOperator&), SetAbsTol(real_t), SetRelTol(real_t), SetDtMax(real_t), SetMPIContext(MPIContext*), bool Step(TimeDependentOperator& op, Vector& state, real_t& t, real_t& dt). **There is no SetTolerances; Step takes the operator as its first argument** (R-004).

Edge Cases

- **Equilibrium verification fails ($> 1e-6$)** at init $\Rightarrow$ abort with the residual (don't proceed with a bad IC). Common cause: τ_{pre}/σ_n sign or frame error — cross-check against CLAUDE.md sign rules.
- **NaN tripwire** in the loop: MPI_Allreduce(MAX) on V_max; abort on NaN/Inf with step/time.
- **use_petsc_ts=true** (optional): route through PetscODESolver/TS with -ts_type rk -ts_rk_type 5dp (matches BP5's PETSc trajectory for the parity cross-check). Gate behind config; default native RK45.

Acceptance Criteria

- ☐ On the BP5 mesh, SetInitialCondition reports equilibrium residual $< 1e-6$.
- ☐ A 50-step BP5 run advances with V_max finite, monotone within the inter-seismic phase, dt growing toward dt_max while locked (sanity per CLAUDE.md regression list).

Dependencies

- Depends on: Phase 4. Required by: Phases 6, 7.

1.18 Phase 6 – I/O: stations/probe, ParaView (fault), checkpoint/restart

Goal

The driver writes on-fault station traces + a global V_max probe, optional fault-surface ParaView, and supports checkpoint/restart of the QD state.

Files to Modify

- drivers/spatial_seas_driver.cpp:
 - **Stations/probe:** reuse io/bp5_parallel_output.hpp (ParallelBP5BenchmarkOutput,

10 BP5 stations) for the BP5-parity runs; add a generic SAF station list (config [output].stations) for SAF Re-solve V from the accepted state at write time (avoid stale RK-stage V).

- **ParaView (fault) — VTU/PVD, NOT VTKHDF (user decision 2026-06-03):** construct `seas::ParaViewOutput<ParMesh>` for the **fault surface** in **FaultOutputMode::Vtu** (`mfem::ParaViewDataCollection`; per-rank `.vtu` + a `.pvd` index), mirroring the dynamic-rupture drivers. **Do NOT use the VTKHDF backend** (`ParaViewHDFDataCollection / .vtkhdf`) even on `MFEM_USE_HDF5` builds — set the QD driver’s default `paraview_fault/paraview_volume` to `"vtu"` (config + CLI `--paraview-fault/--paraview-volume`), overriding the HDF5 default that CLAUDE.md’s Phase-6 R-310 set for the TPV*/BP5 drivers. **Default volume PV off** (honor CLAUDE.md Phase-4 R-313 — volume off unless `--paraview-volume-dt/--volume-pv-dt` set); when on, it is also `Vtu` mode. Adaptive cadence via `AdaptiveSchedule` (coseismic 0.1 s / interseismic 0.1 yr).
- **Checkpoint/restart:** add a **QD-native** checkpoint (e.g. `io/seas_qd_checkpoint.hpp`) that persists (`t`, `dt`, `step`) + the flat **fault state vector** (`[s_dip,s_strike,psi]*N_` owned) + per-DOF static params, with a `driver_tag="spatial_seas"` trailer. **Do NOT reuse `io/tpv104_checkpoint.hpp` (R-009):** it `#includes` `dynamic/fault_face_flux.hpp` (`DOFData`) and its API is `Q+std::vector<DOFData>-centric` (`tpv104_checkpoint.hpp:40,74–77,147–150`) — reusing it **violates §Constraints “No dynamic-code dependency”** (project memory [1]/[C2]) and does not fit the QD state (a flat Vector, **no** bulk Q). You may copy the `Gmsh-v2.2` / per-rank framing + `DRIVER_TAG_V1` trailer *logic*, but not the header itself. On restart, skip nucleation re-seed if already in the checkpoint.

Edge Cases

- Restart safety: refuse if `output-dir == restart parent`; refuse `driver_tag` mismatch (reuse `spatial_dyn_driver.cpp:852–894` logic).
- `--paraview-bulk-*` on this driver: one-time “no effect” warning (no secondary stress collection in QD), per CLAUDE.md Phase-6.4.

Acceptance Criteria

- ☐ Station `.dat` files + probe `.dat` written; columns match the BP5 SCEC format for the parity run.
- ☐ Checkpoint at step N + restart reproduces the un-checkpointed trajectory to round-off for ≥ 10 further steps.

Dependencies

- Depends on: Phase 5. Required by: Phase 7 (parity needs station output).

1.19 Phase 7 – BP5 benchmark parity (the validation gate)

Goal

Running `spatial_seas_driver` on the BP5 problem reproduces `seas_driver.cpp` station output to the existing golden tolerance — proving the spatial→QD wiring is physically correct before any SAF run.

Files to Create

- `config/safs_qd/bp5_spatial_seas_verification.toml` (sibling `config/safs_qd/bp5_spatial_seas_verification_cgmg.toml`) — a `SpatialFrictionConfig` TOML encoding the BP5 problem: `[friction.rate_state]` with BP5 $a(z)/b/Dc/V0/f0$, `[boundary]` BP5 far-field Dirichlet tags (1–4 Dirichlet, 5–6 Natural), `[solver].type="mumps"` (exact cross-check) **and** a sibling `...cgmg.toml` with `cg_amg`. **Prestress (R-007)**: uniform `FaultLocalPrestress` **cannot** reproduce BP5’s prestress, which is spatially heterogeneous — `bp5_params.hpp:333` $\tau_{0_vec} = \sigma_n \cdot a(x2, x3) \cdot \text{asinh}(V_{init}/(2V0)) \cdot e^{\{\psi_{ss}/a\}} + \eta \cdot |V_{init}|$ **plus** a nucleation-zone `delta_tau`. None of the existing stress kinds (`ConstantTensor/FaultLocalPrestress/DepthProportionalToShearModulus/SidecarHDF5`) evaluates that formula, and `ResolveRateState` does not produce `tau_pre` at all (`RateStatePerDOFParams` has no prestress field). So the verification config **MUST** source a **per-DOF** BP5 prestress: either **(i)** a new `bp5_analytic` stress source that fills `tau_pre_`/`sigma_n_per_dof_` by calling `bp5_params::tau0_vec` on the owned fault DOFs (cleanest — mirrors `seas_driver.cpp` exactly), or **(ii)** a precomputed `[stress]` `SidecarHDF5` of per-DOF τ_0/σ_n generated from `bp5_params::tau0_vec`. `FaultLocalPrestress` (uniform) stays valid only for problems whose prestress is genuinely uniform — **not** BP5. (The `bp5_analytic` source is specified in **Phase 3c** — small, additive, on the BP5-parity critical path.)
- `tests/verification/test_spatial_seas_bp5_parity.*` (or a make target) comparing station output to the `seas_driver.cpp` golden (`config/bp5_verification_50step.toml`).

Detailed Requirements

1. Run both `seas_driver` (golden) and `seas_spatial_seas_driver` on the **same BP5 mesh** for the same step budget.
2. Compare station time, `slip_*`, `log10V_*`, `tau_*`, `log10theta` to the existing BP5 golden tolerance.
3. Diagnose any mismatch against `CLAUDE.md` regression criteria (recurrence ~ 240 yr, dip slip ≈ 0 , no $\tau > 1$ GPa).

Acceptance Criteria

- ☐ `mumps` variant matches `seas_driver.cpp` to golden tolerance over the 50-step window.
- ☐ `cg_amg` variant matches the `mumps` variant to $< 1e-6$ (relative) on station traces.
- ☐ `mpirun -np 8` parity holds.

Dependencies

- Depends on: Phases 3–6. Required by: Phase 8 (gate).

1.20 Phase 8 – SAF quasi-dynamic problem + Frontera jobs + scaling

Goal

Define the SAF QD problem (friction, stress, loading, nucleation) on the named 500 m mesh, run a short dev smoke on Frontera, then a production run; measure solver scaling.

Files to Create

- `config/safs_qd/safs_qd_500m.toml` — `[mesh].path = safs/project_7.0_alternative/experimental_mesh_refinement/safs_fault_box_nwcut_500m_lcfar3000_z0embed-triq.msh`; `[friction.rate_state]` depth profile ($a(z)$ VW core / VS borders); `[stress]` (depth-proportional or sidecar); `[boundary]` SAF far-field Dirichlet + $z=0$ Natural free surface; `[material]` velocity model (heterogeneous λ, μ via Phase 2b); `[time]` `dt_max_years=0.1`, plate rate; `[solver].type="cg_amg"`; ParaView in VTU mode.
- `jobs/safs_qd/safs_qd_500m_dev_2hr.sbatch` and `..._production.sbatch` — mirror `jobs/safs/spatial_dyn_*` (build check, `LD_LIBRARY_PATH` HYPRE-shadow fix per project memory, `ibrun ./seas_spatial_seas_driver --config ...`). **QD jobs live under `jobs/safs_qd/`** (distinct from the dynamic-rupture jobs under `jobs/safs/`).

Detailed Requirements / Investigation (specify before coding the config)

1. **SAF boundary loading:** BP5's $u_x = \text{sgn}(Y) * V_p * t / 2$ assumes a planar $y=0$ fault. The SAF fault is **curved/non-planar** — the far-field Dirichlet must impose the regional plate motion consistent with the fault's strike; define the loading function for the SAF box (likely a uniform far-field velocity gradient, not $\text{sgn}(Y)$). **This is a physics design item — confirm with the user / SCEC SAF setup before production.**
2. **SAF nucleation:** choose IC perturbation (elevated V_{init} /reduced $a-b$ patch) location on the SAF surface; resolve into state/ τ_{pre} at $t=0$ (no dynamic gradual overstress).
3. **σ_n model:** depth-proportional effective normal stress with pore-pressure; reuse `[stress.depth_proportional]` + `[stress.pore_pressure]`.
4. **Material:** heterogeneous λ, μ are available (Phase 2b is implemented/required). Configure the SAF velocity model (depth profile or velocity sidecar) via `[material]`; the constant path remains only for BP5 parity.

Acceptance Criteria

- ☐ Dev smoke (short t_{final} , $np \sim 400-800$) on Frontera: builds, initializes (equilibrium $< 1e-6$), advances ≥ 100 steps, V_{max} finite, AMG converging each step.
- ☐ Solver scaling logged: AMG setup + per-solve iteration counts and **wall-times vs np** via a Caliper runtime-report/perfgraph (`MFEM_USE_CALIPER=YES` build); pre-submit size via `scripts/estimate_output_size.py`.
- ☐ Production run reaches an inter-seismic $\rightarrow$ nucleation $\rightarrow$ event sequence with physically plausible recurrence/slip (no CLAUDE.md regression trips).

Dependencies

- Depends on: Phase 7 (gate). Required by: nothing (terminal).
-

1.21 Phase 9 – GPU-readiness (DESIGN ONLY – out of scope this round)

Per decision (B), write **no** device code now. Keep the path GPU-portable so `gpu_dev v1–v3` apply: - Solver: `cg_amg` is the GPU story (link GPU-HYPRE later; BoomerAMG runs on-device on the still-assembled K). MUMPS is a GPU dead-end — do not make it the SAF default. - Coupling kernels (slip-RHS / Dirichlet / traction recovery / friction RHS): keep them in the existing `ElasticityDomainOperator/RateStateFaultOperator` form that `gpu_dev §3.2` targets; don't introduce new host-only `std::vector<DenseMatrix>/std::function` patterns in new code. - Friction: keep Brent on host (CPU) but preserve the `SolveNR` fixed-iteration variant's call shape for a future device path; FP64 only. - Reference: `document/gpu_dev/PA_and_custom_function_gpu_analysis.md §1.5, §6`.

1.22 Testing Strategy

Level	Test	Phase
Unit	<code>test_spatial_seas_config</code> — schema parse + warnings	1
Unit	<code>test_spatial_seas_faultgeom_</code> parity — spatial path == analytic BP5 arrays	3
Unit	heterogeneous-material slab: constant path bit-for-bit + 2-layer traction	2b
Unit	<code>test_spatial_seas_iterative_</code> vs <code>direct</code> — CG-AMG == MUMPS (u + traction)	4
Perf	Caliper perfgraph on small BP5: solver/precond compare + AMG-setup-once	4
Integration	50-step BP5 advance, equilibrium <1e-6, V_max sane	5
I/O	checkpoint→restart round-trip	6
Verification	BP5 station parity vs seas_driver.cpp golden (mumps + cg_amg; np=8)	7
System	SAF dev smoke + production on Frontera; AMG scaling	8

Validation philosophy: **correctness is gated by BP5 parity (Phase 7)**; the iterative solver is gated by direct-solve equivalence (Phase 4); the integration is gated by reproducing the analytic-BP5 FaultGeometry (Phase 3). Each phase compiles, passes make test, and is reviewable before the next.

1.23 Risk Assessment

Risk	Likelihood	Detection	Mitigation
Owned-vs-local fault-DOF seam (spatial ctor uses full-local N; QD stack uses owned N; GetFaultDOFCoords3D is local)	High	Phase-3 parity test (esp. np=4) sizes mismatch / resolver assert	Use the compute_bp5_params=false domain-op ctor (owned layout) and RestrictToOwnedFault(coords, .., 3) before ResolveRateState; assert dof_coords_3d.Size()/3 == dof_to_elem.Size() == GetNumOwnedFaultDOFs(). (R-002)
Spatially-varying b/f0/V0 not representable (QD law is scalar)	Medium	Phase-3 uniformity assert fires	Abort clearly; if SAF needs it, schedule Phase 3b (per-DOF b/f0/V0 in DieterichRuinaFriction/AgingLawPsi + all call sites).
AMG slow / non-convergent for DG elasticity at SAF penalty scales	Medium	Phase-4 iteration count blows up; residual warning	Tune SetElasticityOptions, penalty, strength threshold; fall back to GMRES+AMG or MUMPS on small meshes; consider p-MG (Tandem mg_cheby) as a follow-up — not in scope now.
SAF non-planar loading BC wrong → starved/over-driven nucleation	High	Recurrence/loading-rate sanity (CLAUDE.md v9/v14–15 class)	Phase-8 investigation item; confirm loading function with user before production.
Equilibrium init fails (sign/frame error)	Medium	SetInitialCondition residual > 1e-6 abort	Cross-check every sign against CLAUDE.md; reuse BP5's frame exactly.
Stale .o / no header deps (Makefile)	Medium	Silent ABI SIGABRT	Force-rebuild after editing shared headers (fault_geometry.hpp, elasticity_operator.hpp, spatial_friction.hpp) — project memory.
Heterogeneous-material assembly bug (Phase 2b now required)	Medium	Phase-2b 1-element slab traction test; constant-path bit-for-bit regression guard; BP5 parity (Phase 7) on the constant path	Single shared assembly path (constant ctor forwards ConstantCoefficient); coefficient ctor is its own commit + full elasticity-assembly tests; non-owning coefficient lifetime asserted.
Solver/precond rebuilt per solve (redundant AMG setup) → slow	Medium	Caliper perfgraph: AMG-setup region count > 1; per-step wall-time ~constant high	Build solver + AMG once at SetupSolver; reuse across RK stages/steps (K never changes in QD); only B updates. Assert setup runs once.

Tricky existing code this plan touches

- `fault/rate_state_fault.hpp`, `domain/elasticity_operator.hpp`, `fault/fault_basis.hpp`, `solver/seas_operator.hpp`, `friction/dieterich_ruina.hpp`, `config/bp5_params.hpp` are all “**extreme care**” files (CLAUDE.md). The only *modifications* to them here are **additive** (new getters/ctor/setter); no existing behavior changes. Each such change is its own commit with the relevant `make test-*` run.
-

1.24 Open Questions (resolve before/at the noted phase)

1. **[Phase 8]** SAF far-field loading function for the curved fault — uniform regional velocity gradient vs BP5-style $\text{sgn}(Y)$? (physics decision; needs user/SCEC input).
 2. **[Phase 3b, conditional]** Does the SAF rate-state spec vary $b/f_0/V_0$ spatially? If yes, the QD friction law needs per-DOF support (larger change).
 3. **[Resolved 2026-06-03 — Phase 2b is now required]** Heterogeneous material (velocity model) is implemented in Phase 2b regardless; BP5 parity (Phase 7) still runs the constant path. The specific SAF velocity-model source (depth profile vs sidecar) is a Phase-8 [material] config item.
 4. **[Phase 5]** Native DormandPrinceRK45 vs PETSc TS for production — native is CPU-simple/GPU-portable; PETSc TS reproduces BP5’s exact trajectory. Default native; keep PETSc TS behind a config flag for the parity cross-check.
 5. **[Phase 7]** Confirm the BP5 golden artifact + tolerance to compare against (existing `bp5_verification_50step` golden).
-

1.25 Appendix – exact signatures referenced (verified in-tree)

```
// domain/elasticity_operator.hpp
enum class SolverType { CG_AMG, MUMPS, MUMPS_BLR, GMRES_BlockILU, GMRES_AMG, SUPERLU,
↪ STRUMPACK }; // :46
ElasticityDomainOperator(MeshType&, int order, const ConstitutiveModel&, real_t Vp,
↪ real_t Wf,
    real_t lf, const BoundaryConfig&, DGMethod=BR2, SolverType=MUMPS_BLR, const
↪ DomainConfig&={}); // :106
void Solve(real_t, const Vector& slip_bc, GridFuncType& u) override;
↪ // :201
void ComputeTraction(const GridFuncType& u, const Vector& slip_bc, Vector& tau, Vector*
↪ sn=0); // :204
int GetNumFaultDOFs() const; int GetNumOwnedFaultDOFs() const; int GetNbfPerFace() const;
↪ // :249,250,287

// fault/rate_state_fault.hpp
RateStateFaultOperator(FaultGeometry<MeshType>*, DieterichRuinaFriction*,
↪ StateEvolution*,
    const BP5Params&, MPIContext*&=nullptr);
↪ // :115
```

```

void SetSAFSMode(bool, const Vector* tau_pre_per_dof=0, const Vector* sigma_n_per_dof=0);
↪ // :194
real_t Init(const Vector& tau, Vector& state); real_t ComputeRHS(const Vector&, const
↪ Vector&, Vector&, const Vector*=0);

// solver/seas_operator.hpp
SEASQuasiDynamicOperator(DomainOpType*, FaultOpType*, MPIContext**=nullptr);
↪ // :63
void SetInitialCondition(Vector&); void SetElasticSigmaN(bool); void Mult(const Vector&,
↪ Vector&) const;

// spatial/code/spatial_friction.hpp
struct RateStatePerD0FParams { Vector a,b,Dc,V_init,f_0,V_0,eta,sigma_n_eff; Vector V_w;
↪ }; // :687
RateStatePerD0FParams ResolveRateState(const RateStateBlock&, const Vector& dof_coords_3d,
const Array<int>& dof_to_elem, const Array<int>& dof_to_attr, const MaterialField&,
↪ ParMesh&,
const PorePressureSpec&, const Vector& sigma_n_per_dof) const;
↪ // :702

// fault/fault_geometry.hpp (NEW additions proposed by this plan)
FaultGeometry(DomainOperator<MeshType>&, const BP5Params& seed, MPIContext*, bool
↪ compute_bp5_params);
void SetRateStatePerD0F(const spatial::RateStatePerD0FParams& rs, const mfem::Vector&
↪ init_vel_dir); // R-010

```